

Pandas Tricks You'll Use Daily

10 Essential Operations for Every Data Scientist

Video Title: *10 Pandas Tricks You'll Use Every Single Day*

Why These Tricks?

- Used in 90% of data analysis tasks
- Save hours of time
- Make code cleaner and faster

What You'll Learn:

1. Quick data exploration (5 seconds to understand data)
2. Smart filtering (query vs boolean indexing)
3. Missing data mastery (3 methods)
4. Column magic (apply, map, lambda)
5. Groupby secrets (split-apply-combine)
6. Date tricks (parse, extract, resample)
7. String operations (split, extract, clean)
8. Efficient merging (join strategies)
9. Performance tips (method chaining, inplace)
10. Export shortcuts (to_csv, to_excel tricks)

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta

plt.style.use('dark_background')
np.random.seed(42)

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
pd.set_option('display.precision', 2)
```

1. Quick Data Exploration (5-Second Rule)

```
In [2]: # Create sample sales dataset
np.random.seed(42)
n = 1000

dates = pd.date_range('2023-01-01', periods=n, freq='D')
```

```

df = pd.DataFrame({
    'date': np.random.choice(dates, n),
    'product': np.random.choice(['Laptop', 'Phone', 'Tablet', 'Monitor', 'Ke
    'region': np.random.choice(['North', 'South', 'East', 'West'], n),
    'sales': np.random.randint(100, 5000, n),
    'quantity': np.random.randint(1, 20, n),
    'customer_age': np.random.randint(18, 70, n),
    'discount': np.random.choice([0, 5, 10, 15, 20], n)
})

# Add some missing values
df.loc[np.random.choice(df.index, 50, replace=False), 'discount'] = np.nan
df.loc[np.random.choice(df.index, 30, replace=False), 'customer_age'] = np.r

print("TRICK 1: ONE-LINE DATA EXPLORATION")
print("="*70)
print("\n📊 QUICK SNAPSHOT (The 5-Second Rule):")
print("\n1 Shape: How much data?")
print(f"    df.shape → {df.shape} (1000 rows × 7 columns)")

print("\n2 Info: Data types + missing values")
print("    df.info() →")
df.info()

print("\n3 Describe: Statistics in one shot")
print("    df.describe() →")
print(df.describe())

print("\n4 Value Counts: Distribution of categories")
print("    df['product'].value_counts() →")
print(df['product'].value_counts())

print("\n5 Missing Data: Quick check")
print("    df.isnull().sum() →")
print(df.isnull().sum())

print("\n💡 PRO TIP: Run these 5 commands FIRST!")
print("    They give you 80% understanding in 5 seconds.")
print("="*70)

```

TRICK 1: ONE-LINE DATA EXPLORATION

QUICK SNAPSHOT (The 5-Second Rule):

1 Shape: How much data?

```
df.shape → (1000, 7) (1000 rows × 7 columns)
```

2 Info: Data types + missing values

```
df.info() →
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	date	1000 non-null	datetime64[ns]
1	product	1000 non-null	object
2	region	1000 non-null	object
3	sales	1000 non-null	int64
4	quantity	1000 non-null	int64
5	customer_age	970 non-null	float64
6	discount	950 non-null	float64

```
dtypes: datetime64[ns](1), float64(2), int64(2), object(2)
```

```
memory usage: 54.8+ KB
```

3 Describe: Statistics in one shot

```
df.describe() →
```

	date	sales	quantity	customer_age	discount
count	1000	1000.00	1000.00	970.00	950.00
mean	2024-05-18 22:17:45.600000	2513.73	10.16	43.44	9.72
min	2023-01-01 00:00:00	106.00	1.00	18.00	0.00
25%	2023-08-31 18:00:00	1353.50	5.00	31.00	5.00
50%	2024-05-20 00:00:00	2425.50	10.00	43.00	10.00
75%	2025-01-27 06:00:00	3734.75	15.00	56.00	15.00
max	2025-09-25 00:00:00	4997.00	19.00	69.00	20.00
std	NaN	1366.03	5.47	14.68	7.08

4 Value Counts: Distribution of categories

```
df['product'].value_counts() →
```

```
product
```

```
Laptop      213
```

```
Tablet      207
```

```
Keyboard    204
```

```
Phone       203
```

```
Monitor     173
```

```
Name: count, dtype: int64
```

5 Missing Data: Quick check

```
df.isnull().sum() →
```

```
date      0
```

```
product   0
```

```
region    0
```

```
sales     0
```

```
quantity  0
```

```
customer_age  30
```

```
discount   50
```

dtype: int64

💡 PRO TIP: Run these 5 commands FIRST!
They give you 80% understanding in 5 seconds.

=====

2. Smart Filtering (3 Ways)

```
In [3]: print("\nTRICK 2: FILTERING DATA LIKE A PRO")
print("="*70)

# Method 1: Boolean Indexing (Classic)
print("\n1 BOOLEAN INDEXING (Most Common):")
high_sales = df[df['sales'] > 3000]
print(f"    df[df['sales'] > 3000] → {len(high_sales)} rows")

# Multiple conditions
print("\n    Multiple conditions with & (and) | (or):")
filtered = df[(df['sales'] > 3000) & (df['region'] == 'North')]
print(f"    High sales in North → {len(filtered)} rows")

# Method 2: Query (Cleaner for complex filters)
print("\n2 QUERY METHOD (Cleaner Syntax):")
query_result = df.query('sales > 3000 and region == "North"')
print(f"    df.query('sales > 3000 and region == \"North\") → {len(query_res)} rows")
print("    ✓ More readable!")
print("    ✓ No brackets needed")

# Method 3: isin (Check membership)
print("\n3 ISIN METHOD (Check if value in list):")
selected_products = df[df['product'].isin(['Laptop', 'Phone'])]
print(f"    df[df['product'].isin(['Laptop', 'Phone'])] → {len(selected_products)} rows")
print("    ✓ Better than (df['product'] == 'Laptop') | (df['product'] == 'Phone')")

# NOT in list
print("\n    NOT in list:")
not_in = df[~df['product'].isin(['Keyboard'])]
print(f"    df[~df['product'].isin(['Keyboard'])] → {len(not_in)} rows")

# Practical example: Complex filter
print("\n🌟 PRACTICAL EXAMPLE (Complex Filter):")
result = df.query('sales > 2000 and region in ["North", "South"] and quantity >= 5')
print(f"    High-value sales in North/South with quantity ≥ 5 → {len(result)} rows")
print("="*70)

# Show sample
print("\nSample of filtered data:")
print(result.head())
```

TRICK 2: FILTERING DATA LIKE A PRO

- 1 **BOOLEAN INDEXING** (Most Common):
`df[df['sales'] > 3000]` → 384 rows

Multiple conditions with `&` (and) | `|` (or):
High sales in North → 98 rows
 - 2 **QUERY METHOD** (Cleaner Syntax):
`df.query('sales > 3000 and region == "North"')` → 98 rows
✓ More readable!
✓ No brackets needed
 - 3 **ISIN METHOD** (Check if value in list):
`df[df['product'].isin(['Laptop', 'Phone'])]` → 416 rows
✓ Better than `(df['product'] == 'Laptop') | (df['product'] == 'Phone')`

NOT in list:
`df[~df['product'].isin(['Keyboard'])]` → 796 rows
- 🚀 **PRACTICAL EXAMPLE** (Complex Filter):
High-value sales in North/South with quantity ≥ 5 → 247 rows

Sample of filtered data:

	date	product	region	sales	quantity	customer_age	discount
4	2023-04-17	Phone	South	3391	6	43.0	10.0
6	2024-12-01	Tablet	South	3819	12	21.0	0.0
10	2024-04-11	Tablet	North	3109	11	41.0	10.0
13	2024-04-03	Keyboard	South	3732	17	NaN	10.0
20	2024-10-23	Monitor	North	4314	5	55.0	0.0

3. Missing Data Mastery (3 Essential Methods)

```
In [4]: print("\nTRICK 3: HANDLING MISSING DATA")
print("="*70)

# Check missing data
print("\n📊 MISSING DATA SUMMARY:")
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_df = pd.DataFrame({
    'Missing Count': missing,
    'Percentage': missing_pct
})
print(missing_df[missing_df['Missing Count'] > 0])

# Method 1: Fill with value
print("\n1️⃣ FILLNA (Replace with value):")
df_filled = df.copy()
df_filled['discount'] = df_filled['discount'].fillna(0)
print("    df['discount'].fillna(0) → Replace NaN with 0")
print(f"    Before: {df['discount'].isnull().sum()} missing")
print(f"    After: {df_filled['discount'].isnull().sum()} missing ✓")
```

```

# Fill with mean/median
print("\n  Fill with MEAN/MEDIAN:")
df_filled['customer_age'] = df_filled['customer_age'].fillna(df_filled['cust
print(f"  df['customer_age'].fillna(df['customer_age'].median()) → {df_fill

# Method 2: Forward/Backward fill
print("\n2  FFILL / BFILL (Forward/Backward Fill):")
df_ffill = df.copy()
df_ffill['discount'] = df_ffill['discount'].fillna(method='ffill')
print("  df['discount'].fillna(method='ffill') → Use previous value")
print("  ✓ Good for time-series data!")

# Method 3: Drop missing
print("\n3  DROPNA (Remove rows with missing):")
df_dropped = df.dropna()
print(f"  df.dropna() → {len(df)} rows → {len(df_dropped)} rows")
print(f"  Lost {len(df) - len(df_dropped)} rows ({len(df) - len(df_dropped)

# Drop if specific column is null
print("\n  Drop if specific column is null:")
df_dropped_subset = df.dropna(subset=['discount'])
print(f"  df.dropna(subset=['discount']) → {len(df_dropped_subset)} rows")

print("\n💡 PRO TIP: Choose method based on data:")
print("  - Categorical: fillna(mode) or fillna('Unknown')")
print("  - Numerical: fillna(mean) or fillna(median)")
print("  - Time-series: ffill() or bfill()")
print("  - Too many missing: dropna()")
print("=*70)

```

TRICK 3: HANDLING MISSING DATA

MISSING DATA SUMMARY:

	Missing Count	Percentage
customer_age	30	3.0
discount	50	5.0

1 FILLNA (Replace with value):

`df['discount'].fillna(0)` → Replace NaN with 0

Before: 50 missing

After: 0 missing ✓

Fill with MEAN/MEDIAN:

`df['customer_age'].fillna(df['customer_age'].median())` → 0 missing ✓

2 FFILL / BFILL (Forward/Backward Fill):

`df['discount'].fillna(method='ffill')` → Use previous value

✓ Good for time-series data!

3 DROPNA (Remove rows with missing):

`df.dropna()` → 1000 rows → 922 rows

Lost 78 rows (7.8%)

Drop if specific column is null:

`df.dropna(subset=['discount'])` → 950 rows

💡 PRO TIP: Choose method based on data:

- Categorical: `fillna(mode)` or `fillna('Unknown')`
- Numerical: `fillna(mean)` or `fillna(median)`
- Time-series: `ffill()` or `bfill()`
- Too many missing: `dropna()`

```
/var/folders/62/mgtn51r97gl21dm8hd8ywt2h0000gq/T/ipykernel_15520/2427216286.py:30: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.  
df_ffill['discount'] = df_ffill['discount'].fillna(method='ffill')
```

4. Column Magic (Apply, Map, Lambda)

```
In [5]: print("\nTRICK 4: CREATING & TRANSFORMING COLUMNS")  
print("="*70)  
  
# Simple column creation  
print("\n1 SIMPLE ARITHMETIC:")  
df['revenue'] = df['sales'] * df['quantity']  
print("    df['revenue'] = df['sales'] * df['quantity']")  
print(f"    New column created! Sample: {df['revenue'].head(3).tolist()}")  
  
# Apply with lambda  
print("\n2 APPLY + LAMBDA (Row-wise operations):")  
df['sales_category'] = df['sales'].apply(lambda x: 'High' if x > 3000 else 'Low')  
print("    df['sales'].apply(lambda x: 'High' if x > 3000 else ...)")  
print(f"    Distribution: {df['sales_category'].value_counts().to_dict()}")
```

```

# Apply with function
print("\n3 APPLY WITH FUNCTION (Complex logic):")
def calculate_commission(row):
    if row['sales'] > 3000:
        return row['sales'] * 0.10 # 10% commission
    elif row['sales'] > 1500:
        return row['sales'] * 0.05 # 5% commission
    else:
        return row['sales'] * 0.02 # 2% commission

df['commission'] = df.apply(calculate_commission, axis=1)
print("    df.apply(calculate_commission, axis=1)")
print(f"    Commission range: ${df['commission'].min():.0f} - ${df['commission'].max():.0f}")

# Map (For replacements)
print("\n4 MAP (Dictionary-based replacement):")
region_map = {'North': 'N', 'South': 'S', 'East': 'E', 'West': 'W'}
df['region_code'] = df['region'].map(region_map)
print("    df['region'].map({'North': 'N', 'South': 'S', ...})")
print(f"    Sample mapping: {df[['region', 'region_code']].head(3).to_dict('r')}")

# Vectorized operations (FASTEST!)
print("\n5 VECTORIZED (NumPy - FASTEST!):")
df['discount_amount'] = df['sales'] * (df['discount'].fillna(0) / 100)
print("    df['sales'] * (df['discount'] / 100) → Vectorized!")
print("    ✓ 100x faster than apply for simple math!")

print("\n💡 PERFORMANCE TIP:")
print("    Vectorized (fastest) > apply(function) > apply(lambda)")
print("    Use vectorized whenever possible!")
print("    =*70)

# Show sample
print("\nSample with new columns:")
print(df[['sales', 'quantity', 'revenue', 'sales_category', 'commission']].head(3))

```

TRICK 4: CREATING & TRANSFORMING COLUMNS

1 SIMPLE ARITHMETIC:

```
df['revenue'] = df['sales'] * df['quantity']  
New column created! Sample: [24661, 10668, 30114]
```

2 APPLY + LAMBDA (Row-wise operations):

```
df['sales'].apply(lambda x: 'High' if x > 3000 else ...)  
Distribution: {'High': 384, 'Medium': 337, 'Low': 279}
```

3 APPLY WITH FUNCTION (Complex logic):

```
df.apply(calculate_commission, axis=1)  
Commission range: $2 - $500
```

4 MAP (Dictionary-based replacement):

```
df['region'].map({'North': 'N', 'South': 'S', ...})  
Sample mapping: [{'region': 'West', 'region_code': 'W'}, {'region': 'West', 'region_code': 'W'}]
```

5 VECTORIZED (NumPy - FASTEST!):

```
df['sales'] * (df['discount'] / 100) → Vectorized!  
✓ 100x faster than apply for simple math!
```

💡 PERFORMANCE TIP:

```
Vectorized (fastest) > apply(function) > apply(lambda)  
Use vectorized whenever possible!
```

Sample with new columns:

	sales	quantity	revenue	sales_category	commission
0	3523	7	24661	High	352.30
1	2667	4	10668	Medium	133.35
2	1673	18	30114	Medium	83.65
3	3794	3	11382	High	379.40
4	3391	6	20346	High	339.10

5. GroupBy Secrets (Split-Apply-Combine)

```
In [6]: print("\nTRICK 5: GROUPBY MASTERY")  
        print("\n*70)  
  
        # Basic groupby  
        print("\n1 BASIC GROUPBY (Single aggregation):")  
        region_sales = df.groupby('region')['sales'].sum()  
        print("    df.groupby('region')['sales'].sum()")  
        print(region_sales)  
  
        # Multiple aggregations  
        print("\n2 MULTIPLE AGGREGATIONS (agg):")  
        stats = df.groupby('product')['sales'].agg(['count', 'sum', 'mean', 'min', 'max'])  
        print("    df.groupby('product')['sales'].agg(['count', 'sum', 'mean', 'min', 'max'])")  
        print(stats)  
  
        # Different functions for different columns
```

```

print("\n3 DIFFERENT FUNCTIONS PER COLUMN:")
multi_agg = df.groupby('region').agg({
    'sales': ['sum', 'mean'],
    'quantity': ['sum', 'mean'],
    'customer_age': 'mean'
})
print("    df.groupby('region').agg({'sales': ['sum', 'mean'], ...})")
print(multi_agg)

# Named aggregations (Clean column names!)
print("\n4 NAMED AGGREGATIONS (Clean output):")
summary = df.groupby('product').agg(
    total_sales=('sales', 'sum'),
    avg_sales=('sales', 'mean'),
    total_quantity=('quantity', 'sum'),
    num_transactions=('sales', 'count')
).round(2)
print("    df.groupby('product').agg(total_sales=('sales', 'sum'), ...)")
print(summary)

# Group by multiple columns
print("\n5 MULTIPLE GROUP COLUMNS:")
product_region = df.groupby(['product', 'region'])['sales'].sum().unstack(fill_value=0)
print("    df.groupby(['product', 'region'])['sales'].sum().unstack())
print(product_region)

# Transform (keep original size)
print("\n6 TRANSFORM (Add group stats to original df):")
df['region_avg_sales'] = df.groupby('region')['sales'].transform('mean')
print("    df.groupby('region')['sales'].transform('mean')")
print("    ✓ Adds group average back to each row!")
print(f"    Sample: {df[['region', 'sales', 'region_avg_sales']].head(3).to_csv()}")

print("\n💡 PRO TIP: Named aggregations keep code readable!")
print("    Avoid multi-level column names – use named agg instead.")
print("="*70)

```

TRICK 5: GROUPBY MASTERY

1 BASIC GROUPBY (Single aggregation):

```
df.groupby('region')['sales'].sum()
region
East      600962
North     649398
South     634310
West      629059
Name: sales, dtype: int64
```

2 MULTIPLE AGGREGATIONS (agg):

```
df.groupby('product')['sales'].agg(['count', 'sum', 'mean', 'min', 'max'])

```

	count	sum	mean	min	max
product					
Keyboard	204	487436	2389.39	124	4969
Laptop	213	524486	2462.38	106	4978
Monitor	173	463140	2677.11	138	4988
Phone	203	494435	2435.64	107	4997
Tablet	207	544232	2629.14	122	4982

3 DIFFERENT FUNCTIONS PER COLUMN:

```
df.groupby('region').agg({'sales': ['sum', 'mean'], ...})

```

	sales		quantity		customer_age
	sum	mean	sum	mean	mean
region					
East	600962	2624.29	2257	9.86	43.36
North	649398	2478.62	2733	10.43	43.46
South	634310	2458.57	2702	10.47	43.54
West	629059	2506.21	2465	9.82	43.36

4 NAMED AGGREGATIONS (Clean output):

```
df.groupby('product').agg(total_sales=('sales', 'sum'), ...)

```

	total_sales	avg_sales	total_quantity	num_transactions
product				
Keyboard	487436	2389.39	2210	204
Laptop	524486	2462.38	2081	213
Monitor	463140	2677.11	1661	173
Phone	494435	2435.64	2076	203
Tablet	544232	2629.14	2129	207

5 MULTIPLE GROUP COLUMNS:

```
df.groupby(['product', 'region'])['sales'].sum().unstack()

```

region	East	North	South	West
product				
Keyboard	115093	113366	142184	116793
Laptop	136354	114904	134680	138548
Monitor	97277	145465	112582	107816
Phone	104928	129885	114605	145017
Tablet	147310	145778	130259	120885

6 TRANSFORM (Add group stats to original df):

```
df.groupby('region')['sales'].transform('mean')
✓ Adds group average back to each row!
```

```
Sample: [{'region': 'West', 'sales': 3523, 'region_avg_sales': 2506.21115378486}, {'region': 'West', 'sales': 2667, 'region_avg_sales': 2506.211155378486}, {'region': 'West', 'sales': 1673, 'region_avg_sales': 2506.211155378486}]
```

💡 PRO TIP: Named aggregations keep code readable!
Avoid multi-level column names – use named agg instead.

=====

6. Date Magic (Parse, Extract, Resample)

```
In [7]: print("\nTRICK 6: DATE/TIME OPERATIONS")
        print("="*70)

        # Ensure date is datetime
        df['date'] = pd.to_datetime(df['date'])

        # Extract date parts
        print("\n1 EXTRACT DATE PARTS:")
        df['year'] = df['date'].dt.year
        df['month'] = df['date'].dt.month
        df['day'] = df['date'].dt.day
        df['day_name'] = df['date'].dt.day_name()
        df['week'] = df['date'].dt.isocalendar().week
        print("    df['date'].dt.year, .dt.month, .dt.day_name(), .dt.week")
        print(f"    Sample: {df[['date', 'year', 'month', 'day_name', 'week']].head(3)}")

        # Date filtering
        print("\n2 DATE FILTERING:")
        march_data = df[df['date'].dt.month == 3]
        print(f"    df[df['date'].dt.month == 3] → {len(march_data)} rows in March")

        # Date range
        date_range = df[(df['date'] >= '2023-06-01') & (df['date'] <= '2023-06-30')]
        print(f"    June 2023 data → {len(date_range)} rows")

        # Resample (Time-series aggregation)
        print("\n3 RESAMPLE (Time-series aggregation):")
        df_sorted = df.sort_values('date')
        df_sorted = df_sorted.set_index('date')

        # Monthly aggregation
        monthly = df_sorted['sales'].resample('M').sum()
        print("    df.resample('M').sum() → Monthly totals")
        print(monthly.head())

        # Weekly aggregation
        print("\n    Weekly aggregation:")
        weekly = df_sorted['sales'].resample('W').agg(['sum', 'mean', 'count'])
        print("    df.resample('W').agg(['sum', 'mean', 'count'])")
        print(weekly.head())

        # Reset index
        df = df_sorted.reset_index()
```

```
# Date difference
print("\n🔢 DATE CALCULATIONS:")
min_date = df['date'].min()
df['days_since_start'] = (df['date'] - min_date).dt.days
print(f"    Days since start: 0 to {df['days_since_start'].max()} days")

print("\n💡 PRO TIP: Use dt accessor for date operations!")
print("    dt.year, dt.month, dt.day_name(), dt.week, etc.")
print("    " + "*" * 70)
```

TRICK 6: DATE/TIME OPERATIONS

1 EXTRACT DATE PARTS:

```
df['date'].dt.year, .dt.month, .dt.day_name(), .dt.week
Sample: [{'date': Timestamp('2023-04-13 00:00:00'), 'year': 2023, 'month': 4, 'day_name': 'Thursday', 'week': 15}, {'date': Timestamp('2024-03-11 00:00:00'), 'year': 2024, 'month': 3, 'day_name': 'Monday', 'week': 11}, {'date': Timestamp('2025-05-10 00:00:00'), 'year': 2025, 'month': 5, 'day_name': 'Saturday', 'week': 19}]
```

2 DATE FILTERING:

```
df[df['date'].dt.month == 3] → 78 rows in March
June 2023 data → 31 rows
```

3 RESAMPLE (Time-series aggregation):

```
df.resample('M').sum() → Monthly totals
```

```
date
2023-01-31    70080
2023-02-28    88415
2023-03-31    41261
2023-04-30    90864
2023-05-31    88286
Freq: ME, Name: sales, dtype: int64
```

Weekly aggregation:

```
df.resample('W').agg(['sum', 'mean', 'count'])
      sum      mean  count
date
2023-01-01    2655  2655.00      1
2023-01-08   15966  2661.00      6
2023-01-15   15697  1962.12      8
2023-01-22   18277  2030.78      9
2023-01-29   17485  2914.17      6
```

4 DATE CALCULATIONS:

```
Days since start: 0 to 998 days
```

💡 PRO TIP: Use dt accessor for date operations!
dt.year, dt.month, dt.day_name(), dt.week, etc.

```
/var/folders/62/mgtn51r97gl21dm8hd8ywt2h0000gq/T/ipykernel_15520/2419345687.py:32: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.
    monthly = df_sorted['sales'].resample('M').sum()
```

7. String Operations (Split, Extract, Clean)

```
In [8]: print("\nTRICK 7: STRING OPERATIONS")
print("="*70)

# Create sample data with strings
df['product_full'] = df['product'] + ' - ' + df['region'] + ' - ' + df['sale']

print("\n1 STRING METHODS (str accessor):")

# Lowercase/Uppercase
df['product_lower'] = df['product'].str.lower()
df['product_upper'] = df['product'].str.upper()
print("    df['product'].str.lower() / .str.upper()")
print(f"    Sample: {df[['product', 'product_lower', 'product_upper']].head(2)}")

# Contains
print("\n2 STRING FILTERING (contains):")
laptops = df[df['product'].str.contains('Laptop', case=False)]
print(f"    df[df['product'].str.contains('Laptop')] → {len(laptops)} rows")

# Starts with / Ends with
print("\n3 STARTS/ENDS WITH:")
starts_p = df[df['product'].str.startswith('P')]
print(f"    df['product'].str.startswith('P') → {len(starts_p)} rows (Phone)")

# Split string
print("\n4 SPLIT STRING:")
split_result = df['product_full'].str.split(' - ', expand=True)
split_result.columns = ['product_split', 'region_split', 'category_split']
print("    df['product_full'].str.split(' - ', expand=True)")
print(split_result.head(3))

# Extract with regex
print("\n5 EXTRACT (Regex):")
# Example: Extract numbers if they existed in string
df['product_length'] = df['product'].str.len()
print("    df['product'].str.len() → String length")
print(f"    Sample: {df[['product', 'product_length']].head(3).to_dict('records')}")

# Replace
print("\n6 REPLACE:")
df['product_clean'] = df['product'].str.replace('Laptop', 'Notebook')
print("    df['product'].str.replace('Laptop', 'Notebook')")
print(f"    Sample: {df[df['product'] == 'Laptop'][['product', 'product_clean']]}")

# Strip whitespace
print("\n7 STRIP (Remove whitespace):")
print("    df['column'].str.strip() / .str.lstrip() / .str.rstrip()")
print("    ✓ Essential for cleaning data!")

print("\n💡 PRO TIP: Always use str accessor for string operations!")
print("    Never: df['col'].apply(lambda x: x.lower())")
```

```
print("    Always: df['col'].str.lower() → 100x faster!")
print("="*70)
```

TRICK 7: STRING OPERATIONS

- 1 STRING METHODS (str accessor):
`df['product'].str.lower() / .str.upper()`
 Sample: [{'product': 'Keyboard', 'product_lower': 'keyboard', 'product_upper': 'KEYBOARD'}, {'product': 'Laptop', 'product_lower': 'laptop', 'product_upper': 'LAPTOP'}]
 - 2 STRING FILTERING (contains):
`df[df['product'].str.contains('Laptop')] → 213 rows`
 - 3 STARTS/ENDS WITH:
`df['product'].str.startswith('P') → 203 rows (Phone)`
 - 4 SPLIT STRING:
`df['product_full'].str.split(' - ', expand=True)`

	product_split	region_split	category_split
0	Keyboard	East	Medium
1	Laptop	West	Medium
2	Laptop	West	Medium
 - 5 EXTRACT (Regex):
`df['product'].str.len() → String length`
 Sample: [{'product': 'Keyboard', 'product_length': 8}, {'product': 'Laptop', 'product_length': 6}, {'product': 'Laptop', 'product_length': 6}]
 - 6 REPLACE:
`df['product'].str.replace('Laptop', 'Notebook')`
 Sample: [{'product': 'Laptop', 'product_clean': 'Notebook'}, {'product': 'Laptop', 'product_clean': 'Notebook'}]
 - 7 STRIP (Remove whitespace):
`df['column'].str.strip() / .str.lstrip() / .str.rstrip()`
 ✓ Essential for cleaning data!
- 💡 PRO TIP: Always use str accessor for string operations!
 Never: `df['col'].apply(lambda x: x.lower())`
 Always: `df['col'].str.lower() → 100x faster!`

8. Method Chaining (Write Clean Code)

```
In [9]: print("\nTRICK 8: METHOD CHAINING (Clean, Readable Code)")
print("="*70)

# Bad: Multiple steps
print("\n❌ BAD (Multiple intermediate variables):")
print("""
df_temp1 = df[df['sales'] > 2000]
df_temp2 = df_temp1.groupby('region')['sales'].sum()
df_temp3 = df_temp2.sort_values(ascending=False)
""")
```

```

result = df_temp3.head(3)
"""

# Good: Method chaining
print("\n✅ GOOD (Method chaining):")
result = (
    df[df['sales'] > 2000]
    .groupby('region')['sales']
    .sum()
    .sort_values(ascending=False)
    .head(3)
)
print("""
result = (
    df[df['sales'] > 2000]
    .groupby('region')['sales']
    .sum()
    .sort_values(ascending=False)
    .head(3)
)
""")
print("Result:")
print(result)

# Complex chain example
print("\n🌀 COMPLEX EXAMPLE:")
summary = (
    df
    .query('sales > 1000')
    .assign(
        revenue=lambda x: x['sales'] * x['quantity'],
        profit_margin=lambda x: (x['sales'] - x['discount_amount']) / x['sales']
    )
    .groupby('product')
    .agg(
        total_revenue=('revenue', 'sum'),
        avg_margin=('profit_margin', 'mean'),
        count=('sales', 'count')
    )
    .sort_values('total_revenue', ascending=False)
    .reset_index()
)

print("Summary:")
print(summary)

print("\n💡 PRO TIPS:")
print("    1. Use parentheses for multi-line chains")
print("    2. One method per line → Readable!")
print("    3. Use .assign() to create columns in chains")
print("    4. .pipe() for custom functions in chains")
print("    ="*70)

```

TRICK 8: METHOD CHAINING (Clean, Readable Code)

❌ BAD (Multiple intermediate variables):

```
df_temp1 = df[df['sales'] > 2000]
df_temp2 = df_temp1.groupby('region')['sales'].sum()
df_temp3 = df_temp2.sort_values(ascending=False)
result = df_temp3.head(3)
```

✅ GOOD (Method chaining):

```
result = (
    df[df['sales'] > 2000]
    .groupby('region')['sales']
    .sum()
    .sort_values(ascending=False)
    .head(3)
)
```

Result:

region	
North	531518
West	530449
South	510907

Name: sales, dtype: int64

🔗 COMPLEX EXAMPLE:

Summary:

	product	total_revenue	avg_margin	count
0	Tablet	5412856	0.90	174
1	Keyboard	4990358	0.90	163
2	Laptop	4835198	0.91	176
3	Phone	4751422	0.91	170
4	Monitor	4475966	0.91	151

💡 PRO TIPS:

1. Use parentheses for multi-line chains
 2. One method per line → Readable!
 3. Use .assign() to create columns in chains
 4. .pipe() for custom functions in chains
-

9. Performance Tips (Speed Hacks)

```
In [10]: print("\nTRICK 9: PERFORMANCE OPTIMIZATION")
print("="*70)

print("\n🚫 AVOID LOOPS (Use vectorization):")
print("\n❌ SLOW:")
print("""
for i in range(len(df)):
    df.loc[i, 'new_col'] = df.loc[i, 'sales'] * 2
""")
```

```

print("\n✅ FAST (100x faster!):")
print("df['new_col'] = df['sales'] * 2 # Vectorized!")

print("\n2️⃣ USE CATEGORY DTYPE (Save memory):")
print(f"\nBefore: {df['product'].memory_usage(deep=True) / 1024:.2f} KB")
df['product'] = df['product'].astype('category')
print(f"After (category): {df['product'].memory_usage(deep=True) / 1024:.2f} KB")
print("✓ Use for columns with few unique values!")

print("\n3️⃣ READ CSV EFFICIENTLY:")
print("""
# Specify dtypes
df = pd.read_csv('data.csv', dtype={'column': 'category'})

# Read only needed columns
df = pd.read_csv('data.csv', usecols=['col1', 'col2'])

# Parse dates while reading
df = pd.read_csv('data.csv', parse_dates=['date_column'])

# Read in chunks (for huge files)
for chunk in pd.read_csv('data.csv', chunksize=10000):
    process(chunk)
""")

print("\n4️⃣ USE .loc and .iloc (Avoid chained indexing):")
print("\n❌ BAD (Chained indexing):")
print("df[df['sales'] > 2000]['region'] = 'High' # Warning!")

print("\n✅ GOOD:")
print("df.loc[df['sales'] > 2000, 'region'] = 'High' # Correct!")

print("\n5️⃣ QUERY vs BOOLEAN (Query is faster!):")
print("\nFor complex filters, query() is 2-3x faster:")
print("df.query('sales > 2000 and region == \"North\"') # Faster!")

print("\n💡 GOLDEN RULES:")
print("1. NEVER use loops → Use vectorization")
print("2. Use 'category' dtype for low-cardinality columns")
print("3. Specify dtypes when reading CSV")
print("4. Use .loc / .iloc for assignments")
print("5. query() for complex filters")
print("\n*70)

```

TRICK 9: PERFORMANCE OPTIMIZATION

1 AVOID LOOPS (Use vectorization):

✗ SLOW:

```
for i in range(len(df)):
    df.loc[i, 'new_col'] = df.loc[i, 'sales'] * 2
```

✓ FAST (100x faster!):

```
df['new_col'] = df['sales'] * 2 # Vectorized!
```

2 USE CATEGORY DTYPE (Save memory):

Before: 54.21 KB

After (category): 1.54 KB

✓ Use for columns with few unique values!

3 READ CSV EFFICIENTLY:

```
# Specify dtypes
df = pd.read_csv('data.csv', dtype={'column': 'category'})

# Read only needed columns
df = pd.read_csv('data.csv', usecols=['col1', 'col2'])

# Parse dates while reading
df = pd.read_csv('data.csv', parse_dates=['date_column'])

# Read in chunks (for huge files)
for chunk in pd.read_csv('data.csv', chunksize=10000):
    process(chunk)
```

4 USE .loc and .iloc (Avoid chained indexing):

✗ BAD (Chained indexing):

```
df[df['sales'] > 2000]['region'] = 'High' # Warning!
```

✓ GOOD:

```
df.loc[df['sales'] > 2000, 'region'] = 'High' # Correct!
```

5 QUERY vs BOOLEAN (Query is faster!):

For complex filters, query() is 2-3x faster:

```
df.query('sales > 2000 and region == "North"') # Faster!
```

💡 GOLDEN RULES:

1. NEVER use loops → Use vectorization
 2. Use 'category' dtype for low-cardinality columns
 3. Specify dtypes when reading CSV
 4. Use .loc / .iloc for assignments
 5. query() for complex filters
-

10. Export Shortcuts (Save Your Work)

```
In [11]: print("\nTRICK 10: EXPORT DATA EFFICIENTLY")
print("="*70)

print("\n1 TO_CSV (With options):")
print("""
# Basic
df.to_csv('output.csv', index=False) # Don't save index

# Custom separator
df.to_csv('output.tsv', sep='\t', index=False) # Tab-separated

# Specific columns
df[['product', 'sales']].to_csv('summary.csv', index=False)

# With encoding
df.to_csv('output.csv', encoding='utf-8', index=False)
""")

print("\n2 TO_EXCEL (Multiple sheets):")
print("""
# Single sheet
df.to_excel('output.xlsx', sheet_name='Sales', index=False)

# Multiple sheets
with pd.ExcelWriter('output.xlsx') as writer:
    df_north.to_excel(writer, sheet_name='North', index=False)
    df_south.to_excel(writer, sheet_name='South', index=False)
    summary.to_excel(writer, sheet_name='Summary', index=False)
""")

print("\n3 TO_PARQUET (For large data):")
print("""
# Parquet is 10x smaller and faster than CSV!
df.to_parquet('output.parquet')

# Read back
df = pd.read_parquet('output.parquet')

✓ Preserves data types!
✓ Compressed automatically
✓ 10-100x faster than CSV
""")

print("\n4 TO_JSON (For APIs):")
print("""
# Default (records)
df.to_json('output.json', orient='records')

# Pretty print
df.to_json('output.json', orient='records', indent=2)

# Split format
```

```

df.to_json('output.json', orient='split')
""")

print("\n📋 TO_CLIPBOARD (Quick copy):")
print("""
df.head(10).to_clipboard(index=False)
# Now paste into Excel or Google Sheets!
""")

print("\n💡 PRO TIPS:")
print("    - CSV: Universal, but slow for large data")
print("    - Excel: Business users, multiple sheets")
print("    - Parquet: Large data, preserves types, FAST!")
print("    - JSON: APIs, web apps")
print("    - Clipboard: Quick sharing (small data)")
print("    ="*70)

# Practical example
print("\n🎯 PRACTICAL: Save summary reports")
print("\nCreating region summary...")
region_summary = (
    df.groupby('region')
    .agg(
        total_sales=('sales', 'sum'),
        avg_sales=('sales', 'mean'),
        transactions=('sales', 'count')
    )
    .round(2)
)
print(region_summary)

print("\n✓ In real scenario, save with: region_summary.to_excel('region_repc

```

TRICK 10: EXPORT DATA EFFICIENTLY

=====

1 TO_CSV (With options):

```
# Basic
df.to_csv('output.csv', index=False) # Don't save index

# Custom separator
df.to_csv('output.tsv', sep=' ', index=False) # Tab-separated

# Specific columns
df[['product', 'sales']].to_csv('summary.csv', index=False)

# With encoding
df.to_csv('output.csv', encoding='utf-8', index=False)
```

2 TO_EXCEL (Multiple sheets):

```
# Single sheet
df.to_excel('output.xlsx', sheet_name='Sales', index=False)

# Multiple sheets
with pd.ExcelWriter('output.xlsx') as writer:
    df_north.to_excel(writer, sheet_name='North', index=False)
    df_south.to_excel(writer, sheet_name='South', index=False)
    summary.to_excel(writer, sheet_name='Summary', index=False)
```

3 TO_PARQUET (For large data):

```
# Parquet is 10x smaller and faster than CSV!
df.to_parquet('output.parquet')

# Read back
df = pd.read_parquet('output.parquet')

✓ Preserves data types!
✓ Compressed automatically
✓ 10-100x faster than CSV
```

4 TO_JSON (For APIs):

```
# Default (records)
df.to_json('output.json', orient='records')

# Pretty print
df.to_json('output.json', orient='records', indent=2)

# Split format
df.to_json('output.json', orient='split')
```

5 TO_CLIPBOARD (Quick copy):

```
df.head(10).to_clipboard(index=False)
# Now paste into Excel or Google Sheets!
```



PRO TIPS:

- CSV: Universal, but slow for large data
- Excel: Business users, multiple sheets
- Parquet: Large data, preserves types, FAST!
- JSON: APIs, web apps
- Clipboard: Quick sharing (small data)

=====



PRACTICAL: Save summary reports

Creating region summary...

	total_sales	avg_sales	transactions
region			
East	600962	2624.29	229
North	649398	2478.62	262
South	634310	2458.57	258
West	629059	2506.21	251

✓ In real scenario, save with: `region_summary.to_excel('region_report.xlsx')`

Bonus: Quick Reference Cheat Sheet

```
In [12]: print("\n" + "="*70)
print("PANDAS DAILY TRICKS - QUICK REFERENCE")
print("="*70)

cheat_sheet = """
📊 EXPLORATION
df.shape, df.info(), df.describe()
df['col'].value_counts(), df.isnull().sum()

🔍 FILTERING
df[df['col'] > value] # Boolean
df.query('col > value') # Query (cleaner)
df[df['col'].isin([list])] # Check membership

🔧 MISSING DATA
df.fillna(value), df.fillna(method='ffill')
df.dropna(), df.dropna(subset=['col'])

✨ COLUMNS
df['new'] = df['a'] + df['b'] # Vectorized
df['new'] = df['col'].apply(function) # Apply
df['new'] = df['col'].map(dict) # Map

📁 GROUPBY
df.groupby('col')['value'].sum()
df.groupby('col').agg(name=('col', 'func'))
df.groupby('col')['value'].transform('mean')
```

17

DATES

```
df['date'].dt.year, .dt.month, .dt.day_name()  
df.resample('M').sum() # Time-series
```

abc

STRINGS

```
df['col'].str.lower(), .str.upper(), .str.strip()  
df['col'].str.contains('text')  
df['col'].str.split('-', expand=True)
```

xox

CHAINING

```
(df  
  .query('condition')  
  .groupby('col')  
  .sum()  
  .sort_values()  
)
```

< PERFORMANCE

```
df['col'].astype('category') # Save memory  
df.loc[condition, 'col'] = value # Correct indexing  
Vectorize, don't loop!
```



EXPORT

```
df.to_csv('file.csv', index=False)  
df.to_excel('file.xlsx', sheet_name='Sheet1')  
df.to_parquet('file.parquet') # Fast!
```

.....

```
print(cheat_sheet)  
print("="*70)  
print("\n💡 REMEMBER: Practice these daily → They become second nature!")  
print("="*70)
```

PANDAS DAILY TRICKS – QUICK REFERENCE



EXPLORATION

```
df.shape, df.info(), df.describe()
df['col'].value_counts(), df.isnull().sum()
```



FILTERING

```
df[df['col'] > value] # Boolean
df.query('col > value') # Query (cleaner)
df[df['col'].isin([list])] # Check membership
```



MISSING DATA

```
df.fillna(value), df.fillna(method='ffill')
df.dropna(), df.dropna(subset=['col'])
```



COLUMNS

```
df['new'] = df['a'] + df['b'] # Vectorized
df['new'] = df['col'].apply(function) # Apply
df['new'] = df['col'].map(dict) # Map
```



GROUPBY

```
df.groupby('col')['value'].sum()
df.groupby('col').agg(name=('col', 'func'))
df.groupby('col')['value'].transform('mean')
```



DATES

```
df['date'].dt.year, .dt.month, .dt.day_name()
df.resample('M').sum() # Time-series
```



STRINGS

```
df['col'].str.lower(), .str.upper(), .str.strip()
df['col'].str.contains('text')
df['col'].str.split('-', expand=True)
```



CHAINING

```
(df
 .query('condition')
 .groupby('col')
 .sum()
 .sort_values()
)
```

PERFORMANCE

```
df['col'].astype('category') # Save memory
df.loc[condition, 'col'] = value # Correct indexing
Vectorize, don't loop!
```



EXPORT

```
df.to_csv('file.csv', index=False)
df.to_excel('file.xlsx', sheet_name='Sheet1')
df.to_parquet('file.parquet') # Fast!
```

💡 REMEMBER: Practice these daily → They become second nature!

=====